

Multi-level Machine Learning-Guided Autotuning for Efficient Code Generation on a Deep Learning Accelerator

JooHyoungh Cha

University of Science and Technology
Daejeon, Republic of Korea
jh.cha@etri.re.kr

Munyoung Lee

Electronics and Telecommunications
Research Institute
Daejeon, Republic of Korea
munyounglee@etri.re.kr

Jinse Kwon

Electronics and Telecommunications
Research Institute
Daejeon, Republic of Korea
kwonse@etri.re.kr

Jemin Lee

Electronics and Telecommunications
Research Institute
Daejeon, Republic of Korea
leejaymin@etri.re.kr

Yongin Kwon*

Electronics and Telecommunications
Research Institute
Daejeon, Republic of Korea
yongin.kwon@etri.re.kr

Abstract

The growing complexity of deep learning models necessitates specialized hardware and software optimizations, particularly for deep learning accelerators. While machine learning-based autotuning methods have emerged as a promising solution to reduce manual effort, both template-based and template-free approaches suffer from prolonged tuning times due to the profiling of invalid configurations, which may result in runtime errors. To address this issue, we propose ML²Tuner, a multi-level machine learning-guided autotuning technique designed to improve efficiency and robustness. ML²Tuner introduces two key ideas: (1) a validity prediction model to filter out invalid configurations prior to profiling, and (2) an advanced performance prediction model that leverages hidden features extracted during the compilation process. Experimental results on an extended VTA accelerator demonstrate that ML²Tuner achieves equivalent performance improvements using only 12.3% of the samples required by a TVM-like approach and reduces invalid profiling attempts by an average of 60.8%, highlighting its potential to enhance autotuning performance by filtering out invalid configurations.

CCS Concepts: • Software and its engineering → Just-in-time compilers; Software performance; • Computing methodologies → Machine learning approaches; • Hardware → Emerging architectures.

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.

LCES '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1921-9/25/06

<https://doi.org/10.1145/3735452.3735538>

Keywords: Deep learning accelerator, Auto-tuning, Performance prediction, Machine learning for systems, Hardware-aware optimization

ACM Reference Format:

JooHyoungh Cha, Munyoung Lee, Jinse Kwon, Jemin Lee, and Yongin Kwon. 2025. Multi-level Machine Learning-Guided Autotuning for Efficient Code Generation on a Deep Learning Accelerator. In *Proceedings of the 26th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCES '25)*, June 16–17, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3735452.3735538>

1 Introduction

The increasing complexity of deep learning (DL) models has resulted in a significant increase in computational and memory demands [16], prompting growing attention to research on specialized hardware and software optimizations for efficient operation processing within resource constraints [34]. In addition to widely recognized accelerators like Google's TPU, Nvidia's Tensor Core, and Intel's Gaudi, a variety of DL accelerators are under active development. These accelerators are designed to specialize in DL computations, such as matrix-matrix multiplication. However, to fully leverage the hardware's parallelism and maximize the use of internal memory, significant software optimizations are required.

DL accelerator interfaces are typically divided into two types: high-level operator libraries [2, 3, 11, 22, 39] and low-level libraries [6, 19, 26]. High-level operator libraries execute combinations of low-level code based on predefined rules that depend on operation inputs. While this approach is easy to apply, it has inherent optimization limitations, as it cannot define rules for all possible DL operations. In contrast, directly utilizing low-level libraries or generating machine codes can be challenging for users without in-depth hardware knowledge. DL compilers [28, 40, 43, 47] bridge this gap

by taking DL operation inputs and autonomously generating optimized code.

Recent research on DL compiler has embraced machine learning-based autotuning techniques bypassing the requirement of domain expertise for reducing development time and the need for extensive human resources [14, 29, 53]. Autotuning compensates for the lack of domain knowledge by exploring numerous code configurations and utilizing performance metrics to train a machine learning model for code optimization.

Template-based methods, such as AutoTVM[12], limit the search space, enhancing the chances of generating valid configurations but constraining the discovery of optimal solutions beyond the predefined space. On the other hand, template-free methods such as Ansor [56] investigate a more extensive search space, which expands the possibility of finding better solutions but also increases the risk of generating invalid configurations by considering a wider range of options.

Irrespective of whether a template-based or template-free approach is used, identifying invalid configurations remains a significant challenge. The use of invalid configurations during machine learning model training can lead to reduced efficiency in the autotuning process and degrade the performance of the final optimized code [45, 52].

The use of sophisticated back-end compilers, such as LLVM [27], helps minimize the probability of erroneous or invalid code generation on CPUs and GPUs. However, certain configurations can still lead to the generation of invalid code or sub-optimal performance in LLVM. In contrast, DL accelerators that rely on scratchpad memory and lack the capacity for sophisticated back-end compilers often exhibit erroneous values or encounter runtime errors [5, 24, 31].

To address the issue of prolonged tuning times caused by profiling invalid configurations, we propose ML²Tuner, a multi-level machine learning-guided autotuning technique. While existing DL compilers attempt to reduce the number of invalid configurations, the diversity of hardware and architectures still leads to tuning delays and reduced cost model performance [32, 44]. By incorporating an additional validity prediction model focused solely on predicting configuration validity and training the performance prediction model exclusively with valid configurations, we can achieve more accurate performance predictions. Furthermore, by integrating an advanced performance prediction model that considers hidden features generated during the compilation process.

Overall, our experiments show that ML²Tuner converges to the same performance on ResNet18’s convolution layers using only about 12.3% of the configuration samples that TVM requires. Notably, for some layers (e.g., Conv1 and Conv3), it achieves equivalent performance using just 11.2% and 11.3% of TVM’s sample size, respectively. Moreover, ML²Tuner reduces invalid profiling attempts by 60.8%

on average, significantly reducing time wasted on tuning. These gains stem from the system’s ability to (1) accurately filter out faulty configurations through a dedicated validity model, and (2) refine performance predictions using additional *hidden features* extracted from the compiler. As a result, ML²Tuner more quickly identifies high-performance configurations and avoids error-prone or unproductive regions, demonstrating a clear efficiency advantage over TVM-like approach.

2 Related Work

2.1 Compiler Autotuning

Compiler autotuning techniques aim to automatically determine optimization passes or phase ordering that improve performance. Early systems like Milepost GCC [15] and Insieme [18] leveraged static features and multi-objective search. Evolutionary algorithms [50], Bayesian optimization [9], and regression-based cost models [58] have been widely used to reduce the need for manual tuning. Reinforcement learning (RL) methods [4], collaborative filtering [8], and flag subset selection [57] further reduce the search cost or generalize across programs. While effective, general-purpose autotuners often assume traditional compiler backends, limiting their applicability to custom accelerators.

2.2 Deep Learning Compilers

Modern Deep Learning compilers aim to bridge the gap between high-level deep learning frameworks and low-level hardware execution. Systems such as TVM [12], XLA [47], MLIR [28], Glow [23], TensorRT [38], and OpenVINO [22] translate computation graphs into optimized tensor programs for specific devices.

TVM and MLIR emphasize extensibility and modular intermediate representations to target a wide range of hardware backends. TVM supports both template-based and schedule autotuning, while MLIR provides an infrastructure to develop reusable compiler components. XLA performs ahead-of-time (AOT) compilation integrated with TensorFlow.

Glow and TensorRT specialize in inference optimization. Glow applies operator fusion and memory planning for embedded platforms, whereas TensorRT fuses and quantizes operators for NVIDIA GPUs. OpenVINO supports Intel CPUs and VPUs through graph rewriting and backend plug-ins.

2.3 Autotuning for Deep Learning Compilers

While these compilers enable substantial performance gains, many rely on rule-based heuristics or require expert knowledge to adapt to new hardware or models. To reduce manual effort, Machine Learning(ML)-based autotuning has emerged as a way to automatically explore scheduling configurations and optimize for performance.

AutoTVM [12] restricts the search space using templates, while Ansor [56] expands it via hierarchical representation

and evolutionary search. Techniques like Ansor, ADE-GBRT [49], LoopTune [17], and Felix [54] use learned or gradient-based cost models to navigate large design spaces. Several studies focus on reducing tuning cost: FamilySeer [30], Pruner [42], Fasor [21], and One-Shot Tuner [46] prune search space or eliminate online search. Moses [55] targets mobile deployment via transferable features, and IntelliGen [33] improves memory efficiency via graph and loop optimizations. Other approaches address shape dynamism [36, 37] or sparse tensors [13, 51].

Despite these advances, many techniques still suffer from long tuning times and configuration failures on non-CPU/GPU backends [5, 31]. Prior studies such as AdaTune [29], Dynatune [53], and ARCO [14] attempt to balance tuning time and performance, but lack explicit mechanisms to handle invalid configurations during cost model training. Our approach addresses this gap using a multi-level machine learning strategy that decouples validity prediction from performance estimation.

3 Preliminaries

3.1 Background: Deep Learning Operations

Deep learning workloads primarily consist of tensor operations such as matrix multiplication and convolution. These operations are computationally intensive and exhibit high data reuse, making them ideal candidates for hardware acceleration and software-level optimization.

Matrix Multiplication is the core primitive of many DL operators. A typical matrix multiplication $C = A \times B$ involves three nested loops over input matrices. Optimization strategies include loop tiling, unrolling, and parallel thread distribution. Efficient tiling and memory hierarchy-aware scheduling significantly impact performance, especially on scratchpad-based accelerators.

Convolution, another fundamental operator, can be transformed into matrix multiplication (im2col) or executed directly using sliding window patterns. Key tunable parameters include tile size (height, width, channel), stride, padding, and loop ordering. DL compilers attempt to optimize these configurations to reduce memory footprint and improve compute intensity.

Tiling and Threading define how computations are partitioned and parallelized across hardware threads. Poor tiling may lead to memory bank conflicts or underutilization of compute resources. Similarly, thread-level parallelism must align with available execution units to achieve high throughput.

Memory Management is crucial, especially on accelerators with limited on-chip memory. Scheduling decisions determine the lifetime of intermediate tensors, buffer reuse, and data prefetching. Suboptimal memory access patterns often result in performance degradation or runtime failures.

Therefore, effective tuning of these parameters is essential to unlock the full potential of DL accelerators. However, the large and irregular design space makes manual tuning impractical, motivating the use of ML-based autotuning frameworks.

3.2 Motivation: Invalidity in Code Tuning

We define a *configuration* as a specific combination of compiler-level transformation parameters (e.g., tiling sizes, thread binding, memory layout) that determine how a DL operator is lowered and executed. Each configuration reflects a unique scheduling strategy explored during autotuning.

While ML-based autotuning systems such as TVM have demonstrated strong performance gains, they often generate a large number of invalid configurations, especially on non-traditional backends such as VTA [35]. These invalid configurations stem from hardware-imposed constraints, limited programming flexibility, and context-sensitive execution behavior. As a result, autotuning incurs wasted profiling time and reduced cost model accuracy due to noisy training data.

Table 1. Layer-wise convolution parameters of ResNet18 and invalidity ratios on VTA. Each layer is profiled by exhaustively executing all tunable configurations. $I(H, W, C)$ indicates the input tensor’s height, width, and channel count. $K(N, H, W)$ represents the number of filters (N), and each filter’s height and width. $O(H, W)$ is the output tensor’s spatial dimensions, and P, S denote the padding size and stride. The invalidity ratio reflects the proportion of configurations that fail to compile or execute correctly.

Name	(a) Information				(b) Invalidity Ratio
	$I(H, W, C)$	$K(N, H, W)$	$O(H, W)$	P, S	
Conv 1	56,56,64	64,3,3	56,56	1,1	0.8264
Conv 2	56,56,64	128,1,1	28,28	0,2	0.7966
Conv 3	56,56,64	128,3,3	28,28	1,2	0.8057
Conv 4	28,28,128	128,3,3	28,28	1,1	0.6935
Conv 5	28,28,128	256,1,1	14,14	0,2	0.5249
Conv 6	28,28,128	256,3,3	14,14	1,2	0.5249
Conv 7	14,14,256	256,3,3	14,14	1,1	0.5249
Conv 8	14,14,256	512,1,1	7,7	0,2	0.5047
Conv 9	14,14,256	512,3,3	7,7	1,2	0.5047
Conv 10	7,7,512	512,3,3	7,7	1,1	0.5047

High Invalidity Rate. Table 1 shows that for certain layers in ResNet18, over 80% of tuning configurations on VTA fail to compile or run correctly. These failures stem from architectural constraints such as limited on-chip buffer size and strict memory alignment requirements.

Unlike CPUs or GPUs, accelerators like VTA enforce rigid restrictions on tiling, buffer usage, and instruction formats, which are often invisible to the cost model. As a result, many configurations generated during tuning violate backend-specific assumptions, leading to frequent invalid outcomes that are hard to anticipate.

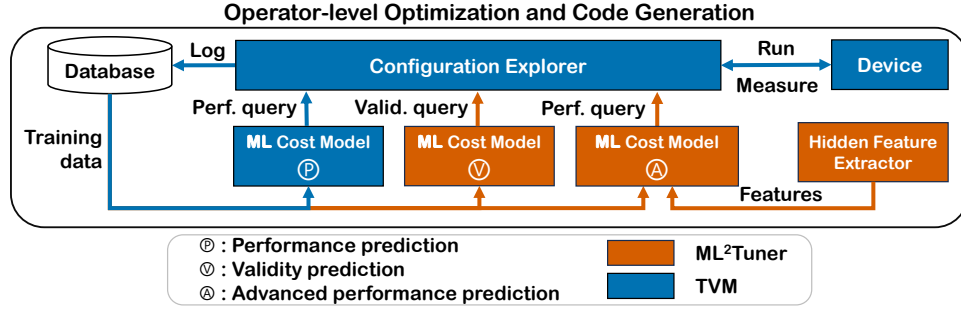


Figure 1. Overview of the automated tuning process of ML²Tuner. Components in blue represent the TVM’s tuning process, while ML²Tuner extends the components highlighted in orange.

Ineffective Learning. Invalid configurations contribute no useful signal to performance prediction models and may mislead the learning process. Moreover, repeated profiling attempts of such configurations increase tuning time without improving performance.

Need for Validity Awareness. To mitigate this, ML²Tuner introduces a dedicated validity prediction model to filter invalid candidates before expensive profiling. Additionally, by training the performance model only on valid samples, ML²Tuner improves prediction accuracy.

This motivates the need for a multi-level machine learning approach that explicitly separates validity prediction from performance estimation, allowing for more efficient and robust autotuning across diverse DL workloads and hardware targets.

4 System Design of ML²Tuner

We present ML²Tuner, an execution configuration tuner that leverages multi-level machine learning to support efficient code optimization on deep learning accelerators. The overall workflow of ML²Tuner is depicted in Figure 1.

In contrast to prior approaches that depend on pre-collected datasets or handcrafted heuristics, ML²Tuner is designed to operate entirely in an online fashion. It initiates the tuning process without any prior knowledge of the target hardware or workload characteristics. Instead, it incrementally profiles configurations, collects data in situ, and trains three machine learning models—P, V, and A—from scratch. This enables ML²Tuner to progressively construct accurate predictors and converge toward optimal configurations.

ML²Tuner is built upon the foundational tuning architecture of TVM, which serves as the basis for both AutoTVM and Ansor. These systems share a common pipeline consisting of a configuration explorer, a cost model, and a profiling database. While preserving this structure, ML²Tuner extends it with two additional models for configuration validity (Model V) and backend-aware performance refinement (Model A), as well as a hidden feature extractor that captures low-level compiler signals. These extensions enable

ML²Tuner to more effectively identify valid configurations and predict their performance with greater precision across diverse DL accelerators.

Configuration Explorer: The configuration explorer selects N code configurations with the highest potential for optimal performance to profile. It begins by defining the search space of all configurations for the given DL operation and assesses their potential based on predictions from the ML models. A code configuration can be a machine code or combination of low-level library calls.

ML Model P: Model P predicts the execution performance of a given configuration using features provided by the configuration explorer. These features are derived from the description of the DL operation—such as input/output tensor sizes and operator types—as well as configuration parameters including tiling factors, thread binding schemes, and memory management strategies, as described in Section 3.1. All of these are visible features that can be extracted without modifying the existing compiler.

While TVM directly selects configurations based solely on Model P’s predictions, ML²Tuner improves this pipeline by incorporating additional validation (Model V) and refinement (Model A) steps.

ML Model V: Model V is dedicated to predicting the validity of configurations, utilizing the same features as Model P. Even if Model P predicts a configuration as highly optimal, ML²Tuner avoids profiling it if Model V predicts it to be invalid. The configuration explorer iteratively applies models P and V until it accumulates $(\alpha + 1) \times N$ configurations. Once enough configurations are collected, ML²Tuner moves on to the next step.

Hidden Feature Extractor: During the compilation of each configuration, ML²Tuner collects internal features generated by the compiler’s static analysis and optimization passes. These include features such as the number and sizes of dummy tiles, the number of dummy threads, and other intermediate indicators that are not visible among the visible features.

Inspired by the feature instrumentation techniques proposed in Mantis [25], our hidden feature extractor automatically collects a large number of backend-specific features during the code lowering process. These include compiler-internal metadata such as the number of branches, loops, method calls, and selected variable values. Due to the large volume of extracted features, ML²Tuner performs feature selection and utilizes only the most important hidden features to augment Model P’s training.

ML Model A: Model A refines performance prediction by incorporating both visible features and hidden features collected during compilation. While Models P and V rely solely on readily available operation and configuration descriptors, Model A leverages additional backend-specific features extracted by the hidden feature extractor, such as dummy tile sizes, internal loop structures, and compiler-internal metadata.

From the $(\alpha + 1) \times N$ configurations initially shortlisted by Models P and V, ML²Tuner compiles each configuration to extract hidden features. Model A then re-evaluates these configurations and selects the final N configurations with the highest predicted performance. This allows ML²Tuner to benefit from low-level architectural insights without requiring manual feature engineering or backend-specific heuristics. By augmenting Model P’s inference with hidden features, Model A improves selection accuracy, especially on accelerators where performance is sensitive to subtle implementation details.

Profiling & Training: The final N configurations selected by Model A are executed on the target hardware. Validity is determined by detecting compilation failures, runtime crashes, or incorrect outputs. For valid configurations, execution times are measured and stored in the database.

Profiling results are used to continuously update the learning models: Model V is trained using the validity outcomes, while Models P and A are refined using the execution times of the valid configurations. This feedback loop improves the predictive accuracy of the models and enables ML²Tuner to adapt more effectively to the characteristics of the target accelerator.

5 Evaluation

5.1 Experimental Setup

Hardware: To evaluate the effectiveness of ML²Tuner, we conducted experiments using Versatile Tensor Accelerator (VTA) [35], an open-source deep learning accelerator architecture designed for deployment on FPGAs. VTA offers a high degree of modularity and configurability, enabling fine-grained control over architectural parameters such as on-chip memory sizes and arithmetic precision. These characteristics make it particularly well-suited for research in hardware-software co-design and compiler autotuning.

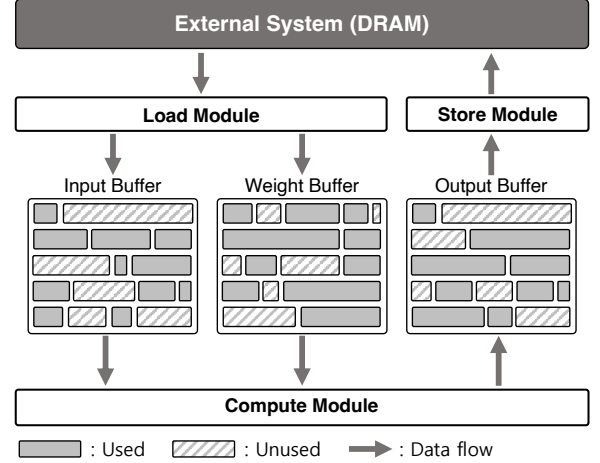


Figure 2. Hardware architecture of VTA. Each buffer (input, weight, output) must dynamically allocate space based on the tile size to store tiled data.

As illustrated in Figure 2, VTA accelerator follows a three-stage pipeline: load, compute, and store whose stages are decoupled via dedicated buffers and task queues. This design facilitates the overlap of memory transfers and computation, thereby enhancing throughput. Optimal configurations minimize memory latency through techniques like double buffering, ensuring high utilization of compute resources. In contrast, poorly tuned configurations can result in misaligned buffer allocations or scheduling conflicts, potentially leading to hardware faults or incorrect outputs. We define such error-prone configurations as *invalid* when targeting VTA.

Table 2. Configuration Parameters of the VTA for FPGA Implementation on Xilinx ZCU102 Board.

Attribute	Description	Value ($\log_2$)
LOG INP WIDTH	Input data type signed integer width.	3
LOG WGT WIDTH	Weight data type signed integer width.	3
LOG ACC WIDTH	Accumulator data type signed integer width.	5
LOG BATCH	VTA matmul intrinsic in/output dimension.	0
LOG BLOCK	VTA matmul inner dimensions.	4
LOG UOP BUFF SIZE	Micro-op on-chip buffer in Bytes.	16
LOG INP BUFF SIZE	Input on-chip buffer in Bytes.	16
LOG WGT BUFF SIZE	Weight on-chip buffer in Bytes.	19
LOG ACC BUFF SIZE	Accumulator on-chip buffer in Bytes.	18

All experiments were conducted on the Xilinx ZCU102 FPGA development board, which features a Zynq UltraScale+ MPSoC. Although TVM provides predefined hardware profiles for other FPGA platforms such as the Xilinx ZCU104, Intel DE10 Nano, Ultra96, and PNYQ, it does not natively support the ZCU102. To adapt VTA for our target platform, we customized its configuration parameters as summarized in Table 2. In particular, we increased the buffer sizes for the UOP, INP, WGT, and ACC components by one level (i.e., incrementing the $\log_2$ -scaled capacity by 1) to more effectively utilize the ample on-chip resources of the ZCU102.

ML²Tuner Implementation: We also implemented the core components of ML²Tuner, including the configuration explorer, hidden feature extractor, and the three ML cost models (P, V, and A), in the PyTorch Glow compiler [23] to compare the performance of ML²Tuner with that of TVM, which uses only the ML cost model P. Additionally, we developed low-level libraries [1] for our VTA and implemented a back-end compiler [40] to execute DL operations by integrating these components. For machine learning-based autotuning, ML²Tuner adopts XGBoost (v2.1.1) [10]. To identify and extract hidden features, ML²Tuner uses an internally integrated compiler. It collects data such as iteration counts from configurations, values affected by conditional expressions, and variations resulting from branch statements. Furthermore, it captures details about the optimization and internal tiling strategies during the code generation process.

Neural Network: Our experiments targeted 10 types of convolution layers from ResNet18 [20], trained on the ImageNet dataset, with the hyperparameters set to $N = 10$ and $\alpha = 1.0$.

Table 1 provides an analysis of the convolution layers of ResNet18 and highlights the ratio of invalid configurations in the VTA. Part (a) provides detailed information about the parameters of each convolution layer, including the input shape, kernel filter, output shape, padding, and stride for each layer. Part (b) presents the invalidity ratio of configurations for each convolution layer.

The optimizable features in our VTA implementation and backend compiler are based on tiling and the number of virtual threads. An invalid configuration is defined as one in which a calculation fails due to a register error, requiring a manual reboot, or a test fails because the result differs from the expected result. Conversely, a valid calculation is a configuration that successfully completes the task without errors.

Hyperparameter Configuration of ML²Tuner Models P and A are designed to identify the highest predicted performance, while Model V is configured for classification tasks. Consequently, Models P, A, and V have distinct objective functions and loss functions, which lead to different search ranges for these parameters during hyperparameter tuning. Models P and A were optimized using regression and ranking objectives to achieve the highest prediction performance. In contrast, Model V was optimized using binary classification objectives to define its search range.

To optimize the hyperparameters of XGBoost, we conducted a grid search [41] based on the search space outlined in Table 3.

As shown in Table 4, Model P and A achieved the highest prediction performance when using Regression objectives, outperforming Rank objectives by 0.06%p in accuracy and being 1.70x faster in computation time. In Model V, the difference in computation time required for different objective functions was greater than the difference in accuracy.

Table 3. Exploration range and final hyperparameter settings for the three XGBoost models used in ML²Tuner. Model P and Model A are trained to predict performance using regression objectives with squared error loss, while Model V is trained for validity classification using a binary hinge objective. The table shows the search space for each hyperparameter and the final selected value for each model after grid search. "reg:S" denotes reg:squarederror, and "b:H" denotes binary:hinge in XGBoost's objective naming.

Parameter	Search Space	Model P	Model V	Model A
objective	Table 4	reg:S	b:H	reg:S
boost round	–	300	300	300
max depth	$\{x \in \mathbb{N} \mid 3 \leq x \leq 15\}$	14	5	14
min child W	$\{x \in \mathbb{R} \mid 1 \leq x < 10\}$	3	3	3
gamma	$\{x \in \mathbb{R} \mid 0.0 \leq x < 1.0\}$	0.0	0.0	0.0
subsample	$\{x \in \mathbb{R} \mid 0.4 \leq x \leq 1.0\}$	1.0	0.6	1.0
colsample BT	$\{x \in \mathbb{R} \mid 0.5 \leq x \leq 1.0\}$	1.0	0.6	1.0
LR	$\{0.001, 0.01, 0.1, 0.2, 0.3\}$	0.01	0.1	0.01
reg alpha	$\{x \in \{-5 \text{ to } 1\} \mid 10^x\}$	10^{-5}	10^{-2}	10^{-5}

Table 4. Comparison of different objective and loss function settings applied to the three models (P, A and V) used in ML²Tuner. The table reports classification accuracy and total tuning time (in seconds) required to explore 200 configurations. Each result corresponds to the best-performing model obtained from $N = 10$ repeated runs with num_boost_round set to 300. Model P and A are trained for performance prediction, while Model V is trained for validity classification.

Model	Objective Function	Loss	Accuracy	Time (sec)
Model P and A	Regression Rank [7]	Squared Error	99.55	320.21
		Logistic	99.49	537.74
Model V	Regression	Squared Error	99.49	316.23
		Logistic	99.47	350.89
	Binary [48]	Hinge Logistic	99.41 99.55	176.73 537.74

5.2 Overall Performance

Figure 3 summarizes the tuning results across all convolution layers (Conv1–Conv10) of ResNet18, comparing ML²Tuner with the baseline TVM autotuner that uses a single performance prediction model.

In part (a), we compare the tuning convergence process. For each layer, we determine the point where TVM's performance plateaued (defined as the same best execution time being repeated more than 10 times), and then evaluate how many samples ML²Tuner required to reach equivalent performance. ML²Tuner consistently converges faster across all layers. For example, it achieved optimal performance using only 11.2% of the samples in Conv1 and 11.3% in Conv3 compared to TVM. On average, ML²Tuner required just 12.3% of the total configuration samples used by TVM across all ten layers.

In part (b), we analyze the invalidity ratio—defined as the proportion of configurations that failed to compile or execute

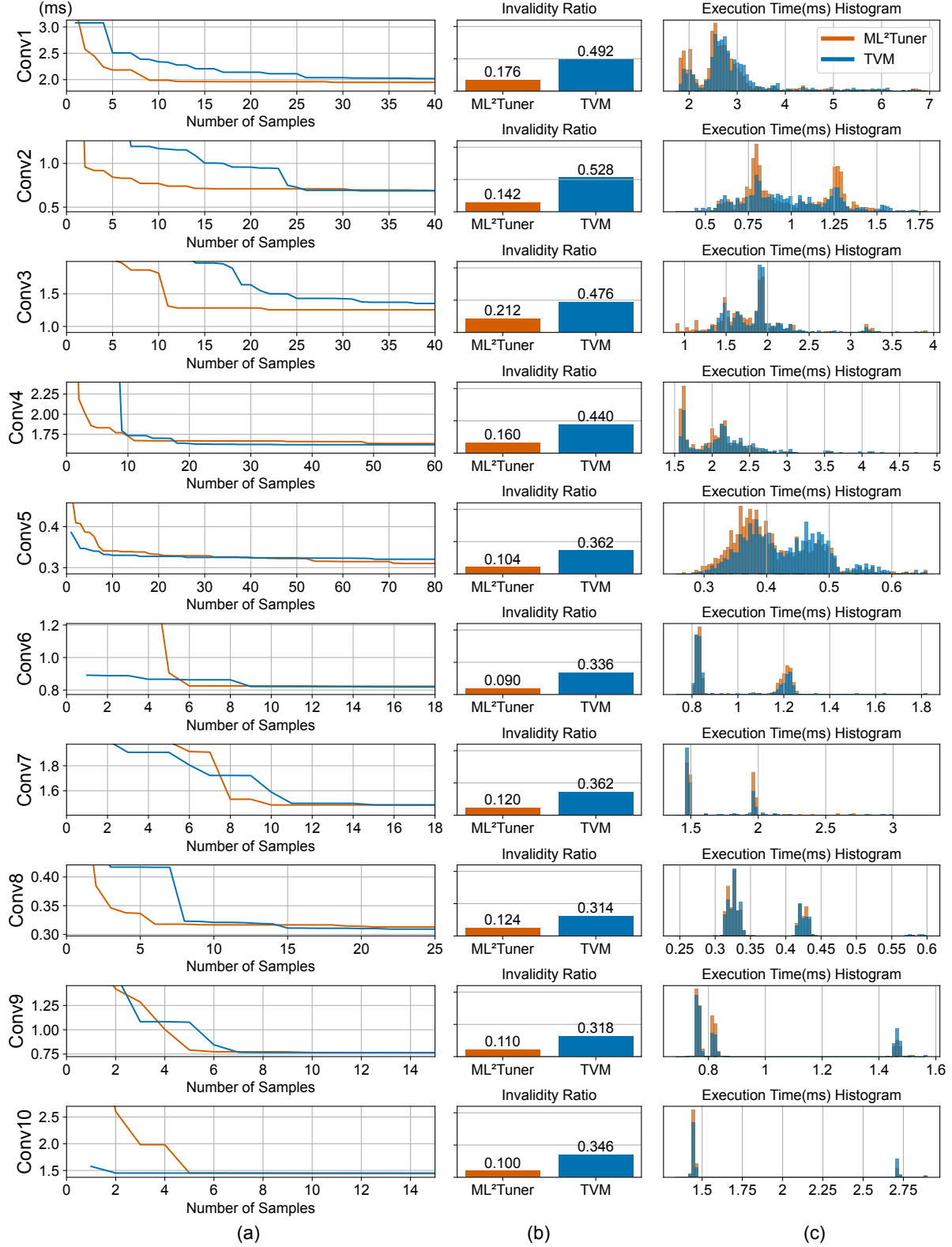


Figure 3. Comparison of tuning results across ResNet18's Conv1 to Conv10 layers. Each row shows results for a single layer, comparing ML²Tuner (orange) and TVM (blue). (a) Execution time (ms) as a function of the number of sampled configurations, where the y-axis indicates the minimum runtime observed so far. (b) Invalidity ratio of configurations selected during the tuning process. (c) Histogram of execution times for valid configurations, illustrating the performance distribution of explored schedules.

Table 5. Importance of Visible and Hidden Features. The **bold** indicates Visible Features that are directly observable, while the non-**bold** denotes information acquired through the Hidden Feature Extractor. The Hidden Feature Extractor outputs two types of variables: (i) those instantiated only along specific control-flow branches (condition-dependent variables), (ii) those derived from regular expressions in the code.

Feature	GeoAVG	Normalized Feature Importance Score (%)									
		Conv1	Conv2	Conv3	Conv4	Conv5	Conv6	Conv7	Conv8	Conv9	Conv10
Tile width	29.268	19.685	27.412	23.810	25.779	32.631	30.633	28.432	31.596	31.975	32.582
Tile height	25.925	15.256	19.768	20.186	22.288	27.235	29.335	29.201	32.406	31.028	30.622
Loop count (Thread creation)	8.468	10.581	8.434	9.058	9.130	6.937	6.231	7.941	8.101	7.579	7.594
Parallelism factor	8.194	5.413	7.907	7.505	7.519	7.965	7.269	7.941	8.912	9.237	8.574
Variable Value (Derived from Num. of kernels and threads)	4.933	7.382	5.271	5.435	4.565	3.854	4.154	3.842	4.321	4.737	3.920
Variable Value (Driven from output size variables)	4.083	2.215	3.163	3.364	3.491	4.625	4.673	4.611	4.861	4.737	4.655
Variable Value (Driven from output size variables)	4.166	2.707	4.217	3.364	3.491	5.139	4.413	4.098	4.051	4.500	4.655
Variable Value (Partial input tile width)	3.069	13.287	6.326	7.764	6.176	1.542	2.336	3.330	0.810	0.711	0.735
Variable Value (Partial output tile height in case A)	1.946	0.738	1.581	1.812	2.685	2.569	2.596	2.561	1.080	1.184	1.715
Loop count (Loop for output store instructions)	1.946	1.722	2.636	2.588	2.954	1.542	2.336	2.305	0.810	0.711	0.980
Variable Value (Resized output tile height in case A)	1.233	9.104	3.426	3.623	1.611	0.257	0.260	0.512	0.270	0.237	0.245
Variable Value (Partial output tile height in case B)	1.069	1.476	1.845	3.623	2.148	0.771	0.779	1.025	0.270	0.237	0.245
Variable Value (Driven from Num. of kernels and threads)	0.740	0.492	0.527	0.518	0.806	1.799	1.038	0.768	0.270	0.474	0.490
Variable Value (Driven from tile width)	0.411	0.246	0.264	0.259	0.269	0.514	0.519	0.256	0.270	0.474	0.490
Variable Value (Resized output tile height in case B)	0.274	0.738	0.791	0.776	1.343	0.257	0.260	0.256	0.027	0.024	0.024
Variable Value (Driven from tile height)	0.384	0.246	0.264	0.259	0.269	0.514	0.519	0.256	0.270	0.474	0.490
Variable Value (Resized input tile height in case A)	0.274	1.230	0.527	0.518	0.215	0.026	0.026	0.077	0.054	0.024	0.024
Variable Value (Resized input tile height in case B)	0.055	0.098	0.105	0.104	0.161	0.026	0.026	0.026	0.001	0.000	0.000

correctly. Random sampling for Conv1 showed a high invalidity ratio of 0.926. TVM’s model-based selection reduced it to 0.492, while ML²Tuner significantly lowered it further to 0.176. This trend persists in other layers; for instance, Conv2 shows a reduction from 0.528 (TVM) to 0.142 (ML²Tuner), and Conv6 drops from 0.336 to 0.090. This demonstrates that ML²Tuner’s dedicated validity predictor (Model V) effectively filters out faulty configurations, reducing wasted profiling time.

Part (c) presents histograms of execution times for valid configurations. Across layers, ML²Tuner’s histograms are more sharply skewed toward lower execution times, whereas TVM’s results are more dispersed. This shows that ML²Tuner not only avoids invalid configurations but also guides the tuning process toward high-performance regions more effectively.

Together, these results confirm that ML²Tuner improves tuning efficiency by accelerating convergence, reducing invalid attempts, and focusing search on promising configurations.

5.3 Impact of Hidden Features

To evaluate how hidden features affect the accuracy of performance prediction, we compared two models: Model P, which uses only visible features, and Model A, which utilizes both visible and compiler-extracted hidden features.

Figure 4 shows the ratio of RMSE values between Model A and Model P across the 10 convolution layers of ResNet18. The Y-axis denotes the RMSE ratio $\text{RMSE}_{\text{Model A}}/\text{RMSE}_{\text{Model P}}$, and the X-axis indicates the number of training samples ranging from 10 to 200. Each bar represents the average of 10 repeated experiments to mitigate variance. Two boosting settings in XGBoost, 100 and 300 rounds, are compared.

The results show that Model A consistently outperforms Model P, achieving an average RMSE ratio of 0.919 across all layers and configurations. This indicates that hidden features help Model A generalize better to unseen configurations. Notably, the benefit becomes more pronounced as the number of boosting rounds increases—highlighting the synergy between strong learners and richer feature representations. For example, in Conv5, the RMSE ratio improves from 0.945 (100 rounds, 200 samples) to 0.854 (300 rounds, 150 samples).

To analyze which features contribute most to model performance, Table 5 reports the normalized feature importance scores across all ResNet18 layers. Among the features, Tiling Width, Tiling Height, and Number of Virtual Threads are explicit configuration parameters directly controlled by the tuner. These parameters define how computations are partitioned and parallelized, forming the core of each schedule configuration explored during autotuning. Their high importance underscores the strong influence of configuration-level decisions on execution performance.

In contrast, hidden features reflect internal compiler derived signals—such as final thread counts, total number of generated loops, output tile sizes, and partial (edge) tile sizes—that are not explicitly included in the configuration parameters but are derived from them. While these features are not directly visible to the configuration search space, they significantly enhance prediction accuracy when incorporated into Model A. By capturing backend-specific behaviors and side effects of scheduling decisions, these hidden features provide valuable insights that are otherwise inaccessible from visible features alone.

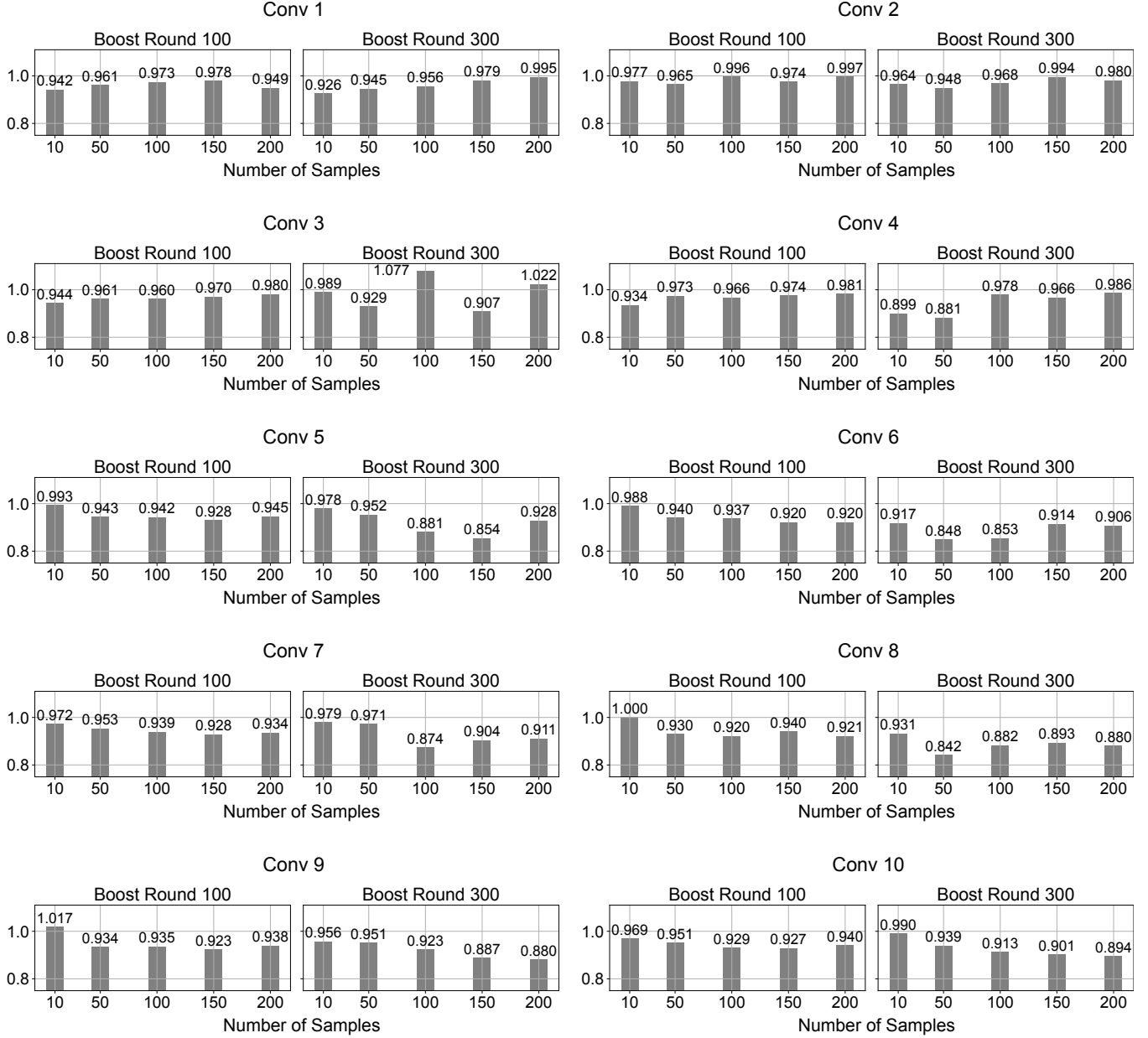


Figure 4. Ratio of RMSE values ($\text{RMSE}_{\text{Model A}}/\text{RMSE}_{\text{Model P}}$) for each ResNet18 layer under different training settings. Each subplot shows results for Boosting Round 100 (left) and 300 (right), with the X-axis indicating the number of training samples (10 to 200). Ratios below 1.0 demonstrate that Model A—leveraging hidden features—achieves more accurate performance predictions than Model P.

Despite the need to compile $(\alpha + 1) \times N$ configurations to extract hidden features, this overhead is justified by the resulting improvement in prediction accuracy and, ultimately, in the quality of selected configurations. ML^2Tuner thus benefits from enhanced performance prediction, which accelerates convergence during tuning and improves final runtime efficiency.

5.4 ML^2Tuner Performance on Diverse Convolutions

To verify the general applicability of ML^2Tuner beyond ResNet based models, we conducted additional experiments on convolution layers from the InceptionV3 model. Unlike ResNets, which primarily employ standard 3×3 and 1×1 kernels, InceptionV3 includes a wider variety of kernel shapes—such as 5×5 , 1×7 , and 7×1 —that introduce distinct scheduling

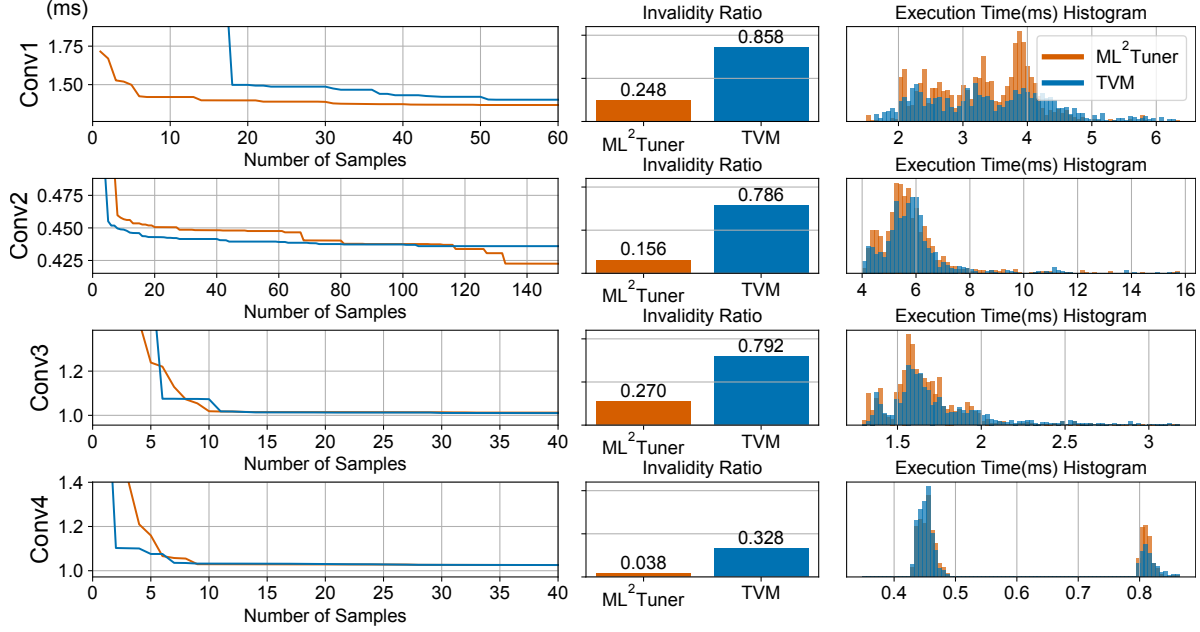


Figure 5. Comparison of tuning results across InceptionV3’s Conv1 to Conv4 layers.

and tiling challenges. We selected four convolution layers from InceptionV3 (Conv1 to Conv4 in Table 6) that exhibit these diverse kernel patterns and evaluated ML²Tuner’s performance.

As shown in Figure 5, ML²Tuner effectively reduces the number of invalid configurations and converges to high-performance schedules with fewer samples. These results confirm ML²Tuner’s effectiveness for diverse convolutions.

Table 6. Layer-wise convolution parameters of InceptionV3 and invalidity ratios on VTA.

Name	(a) Information				(b) Invalidity Ratio
	I(H, W, C)	K(N, H, W)	O(H, W)	P, S	
Conv 1	35,35,48	64,5,5	35,35	2,1	0.7748
Conv 2	8,8,384	384,3,1	14,14	1,1	0.3281
Conv 3	17,17,192	192,1,7	17,17	3,1	0.5172
Conv 4	17,17,192	192,7,1	17,17	3,1	0.5306

6 Conclusion and Future Work

This paper presented ML²Tuner, a multi-level machine learning-guided autotuning framework that improves the efficiency and reliability of deep learning compiler optimization, particularly for hardware platforms with high invalid configuration rates. ML²Tuner separates the tasks of validity filtering and performance prediction using two dedicated models (Model V and Model A), and further enhances prediction accuracy by incorporating hidden features extracted during compilation.

Our experimental results on ResNet18 convolution layers running on a VTA-based accelerator show that ML²Tuner achieves comparable performance to traditional TVM-based tuning while using only 12.3% of the configuration samples. In addition, ML²Tuner reduces invalid profiling attempts by an average of 60.8%, and its hidden feature-based model improves prediction accuracy with an average RMSE ratio of 0.919 compared to the baseline.

In future work, we plan to extend ML²Tuner to a wider variety of hardware backends beyond VTA, including commercial AI accelerators and edge devices. We also aim to explore more sophisticated learning techniques such as Bayesian optimization and reinforcement learning to further improve tuning quality and speed. Lastly, we intend to develop fault-tolerant mechanisms to automatically recover from runtime failures during tuning, increasing robustness and automation of the autotuning process.

Overall, ML²Tuner provides a practical and extensible framework for compiler autotuning, offering both performance and robustness improvements critical to deploying deep learning models efficiently on diverse hardware platforms.

Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2023-00277060, No.RS-2024-00459797, No.RS-2025-02216517, No.RS-2025-02214497)

References

- [1] 2024. *VTA Library customized by ONES AI*. Accessed: 2025-03-21.
- [2] Martin Abadi and et al. 2015. *TensorFlow, Large-scale machine learning on heterogeneous systems*. doi:10.5281/zenodo.4724125
- [3] Jason Ansel and et al. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. ACM. doi:10.1145/3620665.3640366
- [4] Nazim Bendib, Iheb Nassim Aouadj, and Riyadh Baghdadi. 2024. A Reinforcement Learning Environment for Automatic Code Optimization in the MLIR Compiler. arXiv:2409.11068 [cs.LG] <https://arxiv.org/abs/2409.11068>
- [5] Jun Bi, Qi Guo, Xiaqing Li, Yongwei Zhao, Yuanbo Wen, Yuxuan Guo, Enshuai Zhou, Xing Hu, Zidong Du, Ling Li, Huaping Chen, and Tian-shi Chen. 2023. Heron: Automatically Constrained High-Performance Library Generation for Deep Learning Accelerators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 314–328. doi:10.1145/3582016.3582061
- [6] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Nécule, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. JAX: composable transformations of Python+NumPy programs. <http://github.com/google/jax>
- [7] Chris J.C. Burges. 2010. *From RankNet to LambdaRank to LambdaMART: An Overview*. Technical Report MSR-TR-2010-82. <https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/>
- [8] Stefano Cereda, Gianluca Palermo, Paolo Cremonesi, and Stefano Doni. 2020. A Collaborative Filtering Approach for the Automatic Tuning of Compiler Optimisations. In *The 21st ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '20)*. ACM, 15–25. doi:10.1145/3372799.3394361
- [9] Junjie Chen, Ningxin Xu, Peiqi Chen, and Hongyu Zhang. 2021. Efficient Compiler Autotuning via Bayesian Optimization. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 1198–1209. doi:10.1109/ICSE43902.2021.00110
- [10] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 785–794. doi:10.1145/2939672.2939785
- [11] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems. arXiv:1512.01274 [cs.DC] <https://arxiv.org/abs/1512.01274>
- [12] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation*. <https://api.semanticscholar.org/CorpusID:52939079>
- [13] Adhitha Dias, Logan Anderson, Kirshanthan Sundararajah, Artem Pelenitsyn, and Milind Kulkarni. 2024. SparseAuto: An Auto-scheduler for Sparse Tensor Computations using Recursive Loop Nest Restructuring. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 290 (Oct. 2024), 30 pages. doi:10.1145/3689730
- [14] Arya Fayyazi, Mehdi Kamal, and Massoud Pedram. 2024. ARCO: Adaptive Multi-Agent Reinforcement Learning-Based Hardware/Software Co-Optimization Compiler for Improved Performance in DNN Accelerator Design. arXiv:2407.08192 [cs.LG] <https://arxiv.org/abs/2407.08192>
- [15] Grigori Fursin and et al. 2011. Milepost GCC: Machine learning enabled self-tuning compiler. *International Journal of Parallel Programming* (2011).
- [16] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39. doi:10.1109/MM.2024.3373763
- [17] Dejan Grubisic, Bram Wasti, Chris Cummins, John Mellor-Crummey, and Aleksandar Zlateski. 2023. LoopTune: Optimizing Tensor Computations with Reinforcement Learning. *arXiv preprint arXiv:2309.01825* (2023). <https://arxiv.org/abs/2309.01825>
- [18] Philipp Gschwandtner, Juan J. Durillo, and Thomas Fahringer. 2014. Multi-Objective Auto-Tuning with Insieme: Optimization and Trade-Off Analysis for Time, Energy and Resource Usage. In *Euro-Par 2014 Parallel Processing*, Fernando Silva, Inês Dutra, and Vitor Santos Costa (Eds.). Springer International Publishing, Cham, 87–98.
- [19] Gaël Guennebaud, Benoît Jacob, et al. 2010. Eigen v3. <http://eigen.tuxfamily.org>.
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. doi:10.1109/CVPR.2016.90
- [21] Hanxian Huang, Xin Chen, and Jishen Zhao. 2024. Fasor: A Fast Tensor Program Optimization Framework for Efficient DNN Deployment. In *Proceedings of the 38th ACM International Conference on Supercomputing* (Kyoto, Japan) (ICS '24). Association for Computing Machinery, New York, NY, USA, 498–510. doi:10.1145/3650200.3656631
- [22] Intel Corporation. 2023. *OpenVINO Toolkit*. <https://docs.openvino.ai/latest/index.html>.
- [23] Diederik P. Kingma and Prafulla Dhariwal. 2018. Glow: Generative Flow with Invertible 1x1 Convolutions. arXiv:1807.03039 [stat.ML] <https://arxiv.org/abs/1807.03039>
- [24] Shamik Kundu, Suvadeep Banerjee, Arnab Raha, Suriyaprakash Nataraajan, and Kanad Basu. 2024. DiagNNose: Toward Error Localization in Deep Learning Hardware-Based on VTA-TVM Stack. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 1 (2024), 217–229. doi:10.1109/TCAD.2023.3303851
- [25] Yongin Kwon, Sangmin Lee, Hayoon Yi, Donghyun Kwon, Seungjun Yang, Byung-Gon Chun, Ling Huang, Petros Maniatis, Mayur Naik, and Yunheung Paek. 2013. Mantis: Automatic performance prediction for smartphone applications. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*. 297–308.
- [26] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. 2015. Numba: a LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC* (Austin, Texas) (LLVM '15). Association for Computing Machinery, New York, NY, USA, Article 7, 6 pages. doi:10.1145/2833157.2833162
- [27] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (Palo Alto, California) (CGO '04). IEEE Computer Society, USA, 75.
- [28] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Arnaud Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *CGO 2021*.
- [29] Menghao Li, Minjia Zhang, Chi Wang, and Mingqin Li. 2020. AdaTune: adaptive tensor program compilation made efficient. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 1241, 13 pages.
- [30] Mingzhen Li, Shanjun Zhang, Fengwei Yu, Ruihao Gong, Yi Liu, Zhongzhi Luan, and Depei Qian. 2023. Exploiting Subgraph Similarities for Efficient Auto-tuning of Tensor Programs. In *Proceedings of the International Conference on Parallel Processing (ICPP)*.

- doi:10.1145/3605573.3605596
- [31] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 530–543. doi:10.1145/3575693.3575707
 - [32] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 530–543. doi:10.1145/3575693.3575707
 - [33] Zixuan Ma, Haojie Wang, Jia Xing, Liyan Zheng, Chen Zhang, Huanqi Cao, Kezhao Huang, Shizhi Tang, Penghan Wang, and Jidong Zhai. 2023. IntelliGen: A Tensor Compiler for Memory-Intensive Operators via Unified Graph-Level and Instruction-Level Optimizations. *arXiv preprint arXiv:2307.04995* (2023). doi:10.48550/arXiv.2307.04995
 - [34] Ashutosh Mishra, Jaekwang Cha, Hyunbin Park, and Shih Kim. 2023. *Artificial Intelligence and Hardware Accelerators*. Springer.
 - [35] Thierry Moreau, Tianqi Chen, Luis Vega, Jared Roesch, Eddie Yan, Lianmin Zheng, Josh Fromm, Ziheng Jiang, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2019. A Hardware-Software Blueprint for Flexible Deep Learning Specialization. *arXiv:1807.04188* [cs.LG] <https://arxiv.org/abs/1807.04188>
 - [36] Pengyu Mu, Yi Liu, Rui Wang, Zhonghao Sun, Hailong Yang, Zhongzhi Luan, and Depei Qian. 2023. HAOTuner: A Hardware Adaptive Operator Auto-Tuner for Dynamic Shape Tensor Compilers. *IEEE Trans. Comput.* 73, 1 (2023), 1–13. doi:10.1109/TC.2023.3288758
 - [37] Pengyu Mu, Linqun Wei, Yi Liu, and Rui Wang. 2024. FTuner: A Fast Dynamic Shape Tensors Program Auto-Tuner for Deep Learning Compilers. *arXiv preprint arXiv:2407.21418* (2024). <https://arxiv.org/abs/2407.21418>
 - [38] NVIDIA Corporation. 2023. *NVIDIA TensorRT: High Performance Deep Learning Inference*. Version 8.6, <https://developer.nvidia.com/tensorrt>.
 - [39] ONNX Runtime developers. 2018. *ONNX Runtime*. <https://github.com/microsoft/onnxruntime>
 - [40] Jeman Park, Misun Yu, Jinse Kwon, Junmo Park, Jemin Lee, and Yongin Kwon. 2024. NEST-C: A deep learning compiler framework for heterogeneous computing systems with artificial intelligence accelerators. *ETRI Journal* 46, 5 (2024), 851–864. doi:10.4218/etrij.2024-0139 *arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.4218/etrij.2024-0139*
 - [41] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830. <http://jmlr.org/papers/v12/pedregosa11a.html>
 - [42] Liang Qiao, Jun Shi, Xiaoyu Hao, Xi Fang, Sen Zhang, Minfan Zhao, Ziqi Zhu, Junshi Chen, Hong An, Xulong Tang, Bing Li, Honghui Yuan, and Xinyang Wang. 2025. Pruner: A Draft-then-Verify Exploration Mechanism to Accelerate Tensor Program Tuning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 949–965. doi:10.1145/3676641.3716269
 - [43] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *SIGPLAN Not.* 48, 6 (jun 2013), 519–530. doi:10.1145/2499370.2462176
 - [44] Dennis Rieber, Moritz Reiber, Oliver Bringmann, and Holger Fröning. 2022. HW-Aware Initialization of DNN Auto-Tuning to Improve Exploration Time and Robustness. *arXiv:2205.15568* [cs.LG] <https://arxiv.org/abs/2205.15568>
 - [45] Jaehun Ryu, Eunhyeok Park, and Hyojin Sung. 2022. One-shot tuner for deep learning compilers. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction* (Seoul, South Korea) (CC 2022). Association for Computing Machinery, New York, NY, USA, 89–103. doi:10.1145/3497776.3517774
 - [46] Jaehun Ryu, Eunhyeok Park, and Hyojin Sung. 2022. One-shot tuner for deep learning compilers. In *Proceedings of the International Conference on Compiler Construction* (CC). doi:10.1145/3497776.3517774
 - [47] Amit Sabne. 2020. XLA : Compiling Machine Learning for Peak Performance.
 - [48] Chen Wang, Chengyuan Deng, and Suzhen Wang. 2020. Imbalance-XGBoost: leveraging weighted and focal losses for binary label-imbalanced classification with XGBoost. *Pattern Recognition Letters* 136 (06 2020). doi:10.1016/j.patrec.2020.05.035
 - [49] Yuyang Wang, Xuxin Lin, Mingwei Zhou, and Yao Liang. 2023. Optimizing Tensor Programs of DNNs via Adaptive Differential Evolution with Hardware Measurement Estimation. *SSRN preprint*. doi:10.2139/ssrn.4528511
 - [50] Guojian Xiao, Siyuan Qin, Kuan Li, Juan Chen, and Jianping Yin. 2024. EA tuner: Comparative Study of Evolutionary Algorithms for Compiler Auto-tuning. In *2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. 419–426. doi:10.1109/CSCWD61410.2024.10580120
 - [51] Bobby Yan, Alexander J. Root, Trevor Gale, David Broman, and Fredrik Kjolstad. 2024. Scorch: A Library for Sparse Deep Learning. *arXiv:2405.16883* [cs.LG] <https://arxiv.org/abs/2405.16883>
 - [52] Xi Zeng, Tian Zhi, Zidong Du, Qi Guo, Ninghui Sun, and Yunji Chen. 2020. ALT: Optimizing Tensor Compilation in Deep Learning Compilers with Active Learning. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*. 623–630. doi:10.1109/ICCD50377.2020.00108
 - [53] Minjia Zhang, Menghao Li, Chi Wang, and Mingqin Li. 2021. DynaTune: Dynamic Tensor Program Optimization in Deep Neural Network Compilation. In *International Conference on Learning Representations*. https://openreview.net/forum?id=GTG63M_KcUI
 - [54] Yifan Zhao, Hashim Sharif, Vikram S. Adve, and Sasa Misailovic. 2024. Felix: Optimizing Tensor Programs with Gradient Descent. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*. <https://misailo.cs.illinois.edu/papers/felix-aspl0s24.pdf> to appear.
 - [55] Zhihe Zhao, Xian Shuai, Neiven Ling, Nan Guan, Zhenyu Yan, and Guoliang Xing. 2023. Moses: Exploiting Cross-Device Transferable Features for On-Device Tensor Program Optimization. In *Proceedings of the 24th International Workshop on Mobile Computing Systems and Applications (HotMobile '23)*. 3–9. doi:10.1145/3572864.3580330
 - [56] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 20)*. 863–879.
 - [57] Mingxuan Zhu and Dan Hao. 2023. Compiler Auto-Tuning via Critical Flag Selection. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1000–1011. doi:10.1109/ASE56229.2023.00209
 - [58] Xiaoyang Zhu and Junjie Hao. 2024. CompTuner++: Fast Compiler Tuning via Lightweight ML Models. In *International Conference on Programming Language Design and Implementation (PLDI)*.

Received 2025-03-21; accepted 2025-04-21